HARBOR VIEW HOTEL RESERVATION SYSTEM (HVHRS)

Midterm Examination — UML Modeling

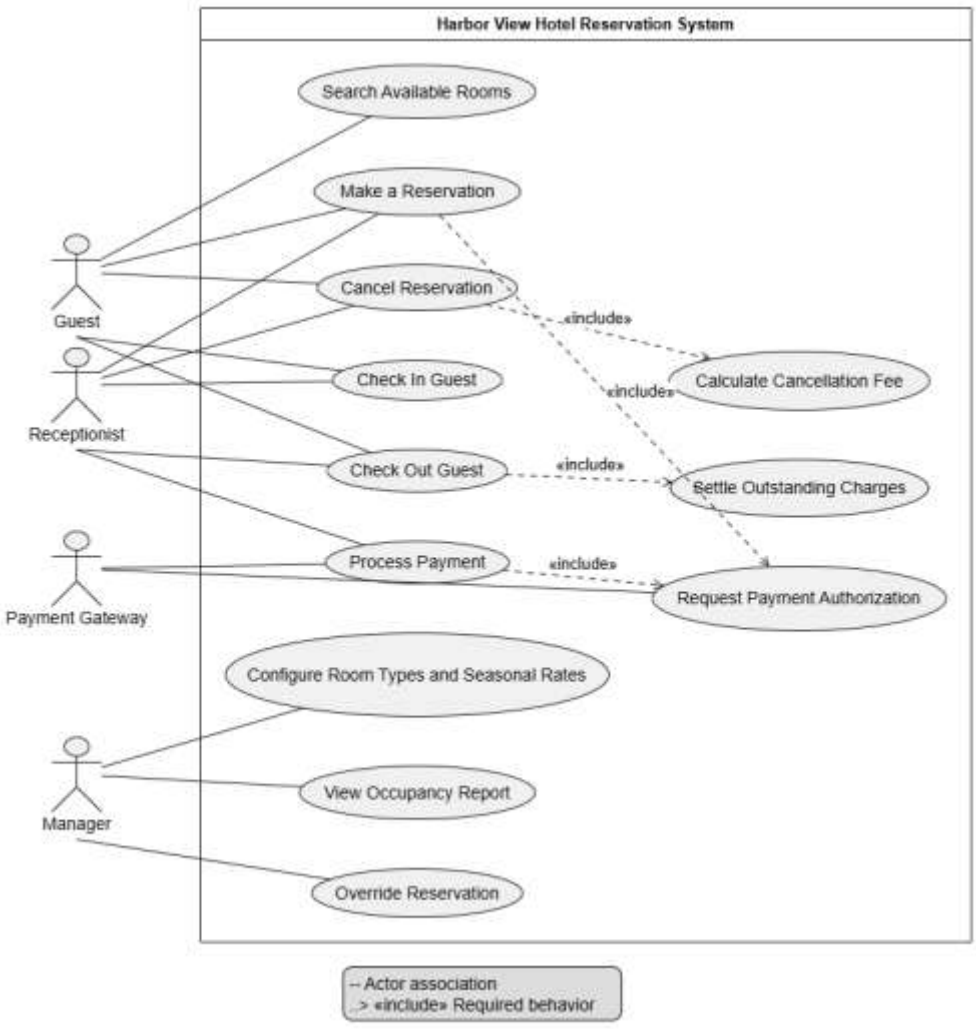
Student Name: MARVIN M. MENESES

Course/Section: BSIT – PD312

Date: October 21, 2026

TASK 1 — USE CASE DIAGRAM

Diagram:



Legend

- **Actor:** A person or external system that interacts with HVHRS.
- **System boundary:** The rectangle enclosing the system's use cases.
- **Association:** A solid line connecting an actor to a use case.
- **<<include>>:** Indicates required behavior reused by another use case.
- **<<extend>>:** Indicates optional or conditional behavior added to a base use case.

The Payment Gateway is an external actor because it authorizes credit-card payments. The Guest, Receptionist, and Manager interact with the system according to their assigned responsibilities.

TASK 2 — FULLY-DRESSED USE CASE DESCRIPTION

Use Case Name

Make a Reservation

Primary Actor: Guest

Supporting Actors: Receptionist (for assisted bookings), Payment Gateway

Goal: Create a reservation for one or more available rooms and confirm it after successful payment authorization.

Trigger

The guest decides to book accommodation after searching for available rooms.

Preconditions

1. The hotel reservation system is operational.
2. The guest can provide the required personal details, stay dates, room selection, and payment information.
3. The system can access room availability and applicable room rates.

Postconditions

On success:

1. A reservation exists with a unique confirmation number.
2. The reservation is associated with exactly one guest and one or more rooms.
3. An authorized payment is recorded.
4. The reservation enters the Confirmed state.
5. Confirmation details are sent to the guest.

On failure:

1. If payment is declined, the reservation remains in the Created state.
2. The guest is notified of the failed authorization.
3. A room that becomes unavailable is not assigned to a confirmed reservation.

Main Success Scenario

1. The guest enters the intended check-in and check-out dates and selects a room type.
2. The system searches for available rooms and displays their nightly rates.
3. The guest selects one or more available rooms.
4. The guest provides personal details and payment information.
5. The system checks room availability again before proceeding.
6. The system creates a reservation in the Created state and generates a unique confirmation number.
7. The system calculates the reservation amount using the applicable room rates.
8. The system creates a payment record associated with the reservation.
9. The system requests payment authorization from the Payment Gateway.

10. The Payment Gateway returns an Authorized result.
11. The system records the authorized payment and changes the reservation state to Confirmed.
12. The system sends the confirmation number and reservation details to the guest.

Alternate Flows

A1 — Payment Declined

1. At Step 10, the Payment Gateway returns Declined.
2. The system records the declined payment.
3. The reservation remains in the Created state.
4. The system notifies the guest that payment authorization failed.
5. The guest may retry with valid payment information.

A2 — Room Becomes Unavailable

1. At Step 5, the system discovers that the selected room is no longer available.
2. The system does not proceed with confirming the requested room.
3. The system informs the guest and displays available alternatives.
4. The guest may select another room and retry.

A3 — Guest Cancels Before Confirmation

1. While the reservation is in the Created state, the guest requests cancellation.
2. The system verifies that cancellation is permitted.
3. The reservation transitions to the Canceled state.
4. The system notifies the guest.

Business Rules

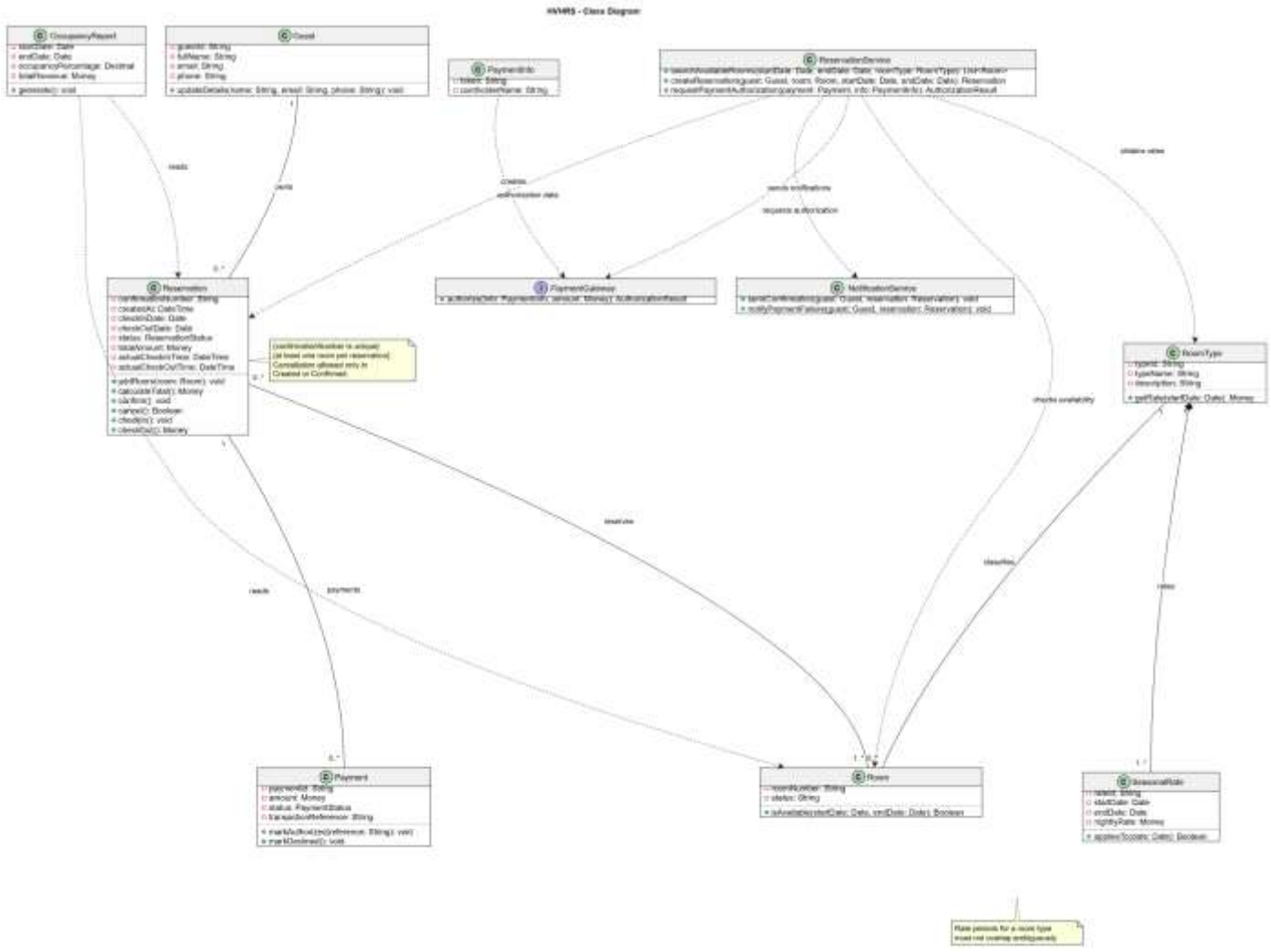
1. Every reservation belongs to exactly one guest.
2. Every reservation contains one or more rooms.
3. A payment belongs to exactly one reservation; a reservation may have multiple payments.
4. Confirmation numbers must be unique.
5. A reservation may be canceled only in the Created or Confirmed state.
6. Cancellation is prohibited after check-in.
7. A declined payment must not automatically confirm a reservation.
8. Seasonal rates and rate-plan rules determine the reservation amount and any applicable cancellation fee.

Non-Functional Requirements

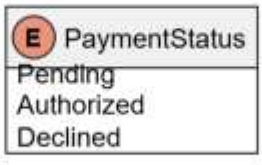
1. Reliability: Reservation and payment records must remain accurate.
2. Security: Personal and payment information must be protected from unauthorized access.
3. Performance: Room searches and booking operations should respond within a reasonable time.
4. Consistency: Room availability must be checked again before booking confirmation to reduce conflicting reservations.

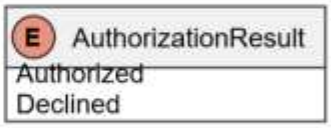
TASK 3 — CLASS DIAGRAM

Diagram:









Design Decisions

The Guest, Room, RoomType, Reservation, and Payment classes represent the main domain concepts.

The Reservation class connects one guest with one or more rooms and maintains the reservation's lifecycle state. Each Room belongs to exactly one RoomType, which provides the room category and associated rate information.

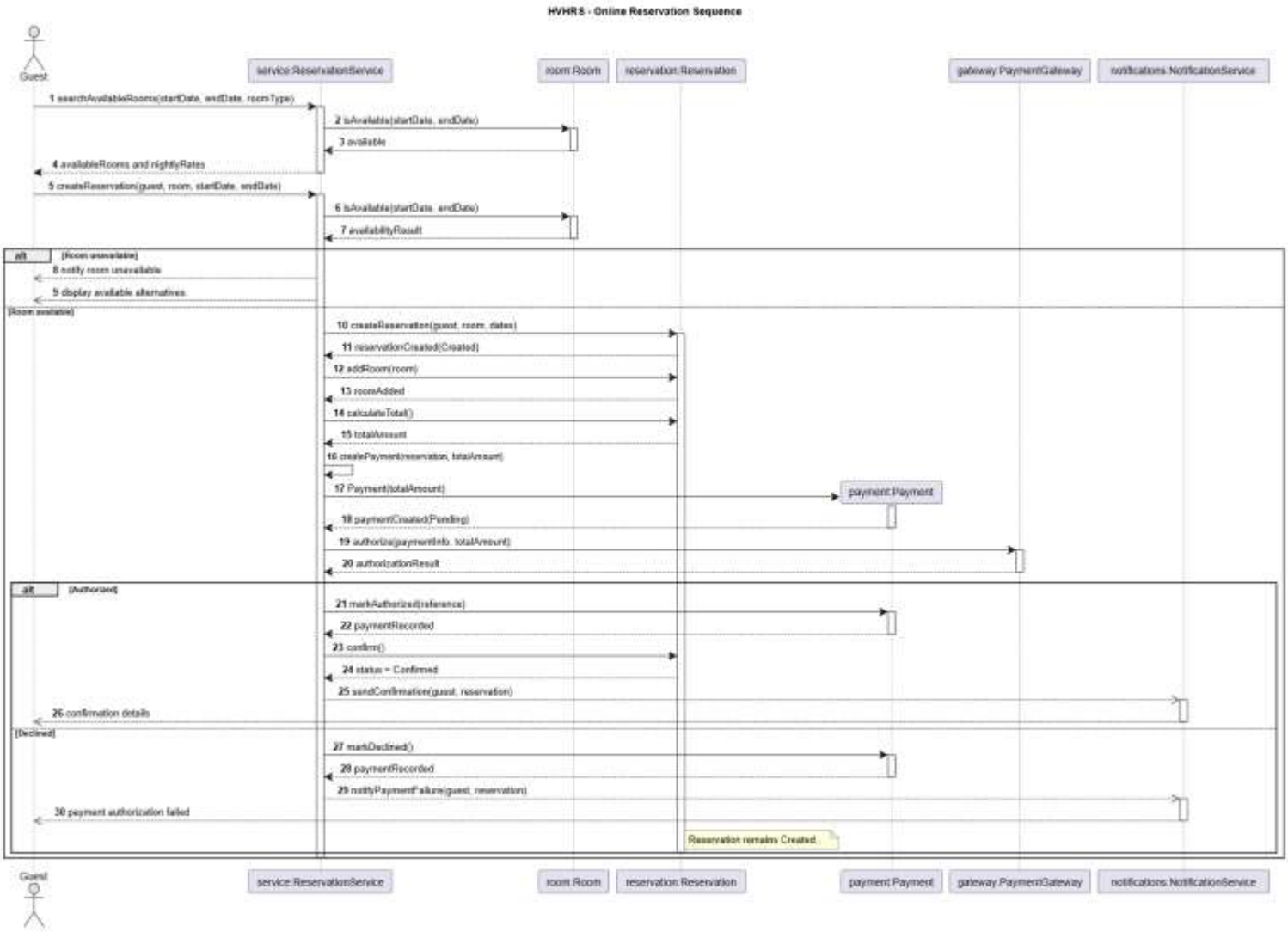
The Payment class is separate from Reservation because one reservation may have multiple payments, such as a deposit and final settlement. Each payment is associated with exactly one reservation.

A ReservationService can coordinate availability checks, reservation creation, payment authorization, and confirmation. A Payment Gateway interface or external participant handles authorization requests without requiring the reservation domain to implement the external payment provider.

Seasonal rates should be represented through a rate-plan or seasonal-rate model if the class diagram includes that supporting class. Confirmation numbers must be unique, and the reservation-to-room association must enforce at least one room per reservation.

TASK 4 — SEQUENCE DIAGRAM

Diagram:



Explanation of Message Flow

The Guest first requests available rooms from ReservationService. The service checks room availability through Room and returns the available rooms and rates.

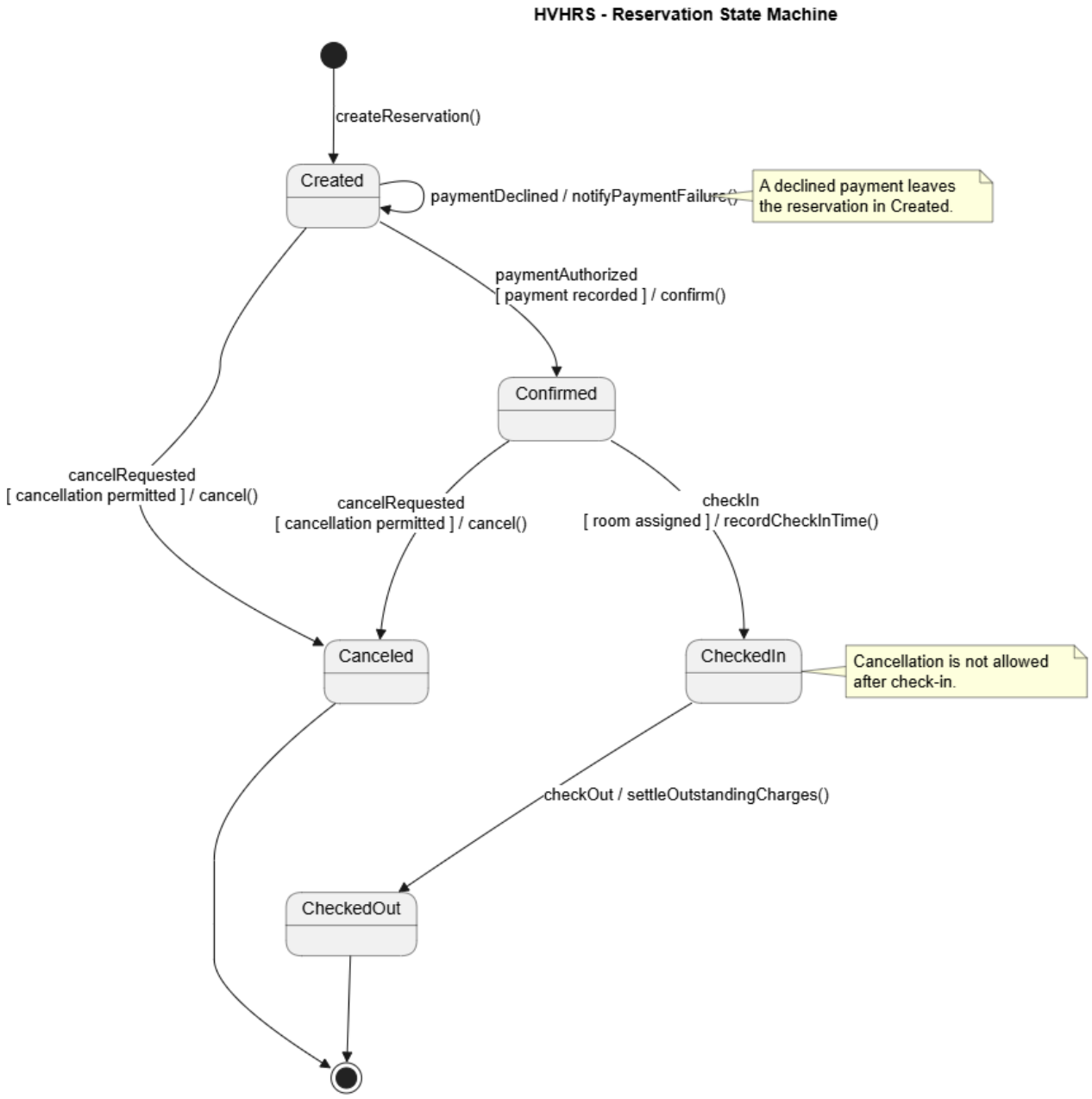
After the guest selects a room and submits personal and payment details, the service checks availability again. If the room is available, the service creates a Reservation, creates a Payment record, and sends an authorization request to the Payment Gateway.

An alt combined fragment represents the two possible authorization outcomes:

- **Authorized:** The service records the authorized payment, confirms the reservation, and sends confirmation details to the guest.
- **Declined:** The service records the declined payment, leaves the reservation in the Created state, and notifies the guest.

A separate room-availability alternative prevents the booking from proceeding when the selected room is no longer available.

TASK 5 — STATE MACHINE DIAGRAM



Explanation of Reservation States

- **Created:** The reservation has been created but has not yet been confirmed.
- **Confirmed:** The reservation has been successfully confirmed after payment authorization.
- **CheckedIn:** The guest has checked in, and the actual check-in time is recorded.
- **CheckedOut:** The guest has checked out, and outstanding charges have been settled.
- **Canceled:** The reservation has been canceled before check-in.

The initial pseudostate leads to Created. Successful payment authorization permits the transition from Created to Confirmed. A declined payment leaves the reservation in Created.

A reservation may transition from Created or Confirmed to Canceled, provided cancellation is permitted. Check-in moves a Confirmed reservation to CheckedIn, and check-out moves it to CheckedOut. CheckedOut is terminal and has no outgoing business-state transitions.

TASK 6 — MODEL CONSISTENCY AND DESIGN RATIONALE

6.1 Explain how your class diagram supports the "Make a Reservation" use case. Identify the classes and associations that collaborate to realise the main success scenario.

The class diagram shows how the system works when a guest books a room. The Guest provides personal details and chooses a room. The ReservationService checks if the room is available and creates a Reservation. The Payment class records the payment, while the Payment Gateway checks if the payment is approved.

If the payment is approved, the reservation becomes Confirmed, and the system sends confirmation details to the guest. If the payment is declined, the reservation stays in Created, and the guest is notified.

Each class has its own job, which helps the system manage bookings correctly.

6.2 Examine your use case diagram, class diagram, sequence diagram, and state machine diagram. Identify one potential inconsistency between any two diagrams (e.g., a message in the sequence diagram that does not correspond to an operation in the class diagram, or a state in the state machine that cannot be reached given the use case description). Describe the inconsistency and propose a correction.

6.2 One Possible Inconsistency Between Diagrams

One possible inconsistency is that the Sequence Diagram shows ReservationService calling createReservation() on the Reservation object, but the Class Diagram lists createReservation() under ReservationService.

To fix this, the Sequence Diagram should show ReservationService creating a new Reservation object instead of calling createReservation() on it. This makes the Sequence Diagram match the Class Diagram and keeps each class's responsibility clear.

6.3 Choose one object-oriented design principle or pattern (e.g., GRASP Creator, GRASP Information Expert, Low Coupling, High Cohesion, or Strategy) that you applied—or could have applied—in your class diagram. Explain:

What it means: The Information Expert principle means that a class should do a job when it has the information needed to do that job.

How it is used: In our system, Reservation manages its own information, such as its rooms, dates, total amount, and status. It can calculate the reservation total using the room rates provided by RoomType and SeasonalRate.

Why it is useful: This makes the system easier to understand and maintain. Each class has a clear responsibility, and the system does not need one class to do everything. If the booking rules change, the related class can be updated without changing the whole system.

CONCLUSION

The four UML diagrams provide complementary views of the Harbor View Hotel Reservation System. The use case diagram describes the system's functionality, the class diagram represents its structure, the sequence diagram shows the interaction flow, and the state machine describes the reservation lifecycle.

Consistency across these models helps ensure that reservation creation, payment authorization, cancellation, check-in, and check-out follow the stated business rules.